

```

objectMapper.valueToTree(ServiceContextHolder.getInstance().getHeader());
headerNode.remove(DTO_LOGICAL_NAME);
responseBody.set(HEADER, headerNode);
}

/* Response Body */
if (body instanceof NH_NIS_ERR_DTO) {
    NH_NIS_ERR_DTOMsgJson json = new NH_NIS_ERR_DTOMsgJson();
    try {
        byte[] errorJson = json.marshal((NH_NIS_ERR_DTO)body);
        responseBody.set(DTO, objectMapper.readTree(errorJson));
    } catch (MarshalException | IOException e) {
        responseBody.set(DTO, objectMapper.valueToTree(body));
    }
} else {
    JsonNode bodyNode = objectMapper.valueToTree(body);

    // 최상위 및 중첩된 모든 DTO 에서 불필요한 필드 제거
    removeUnwantedFields(bodyNode);

    responseBody.set(DTO, bodyNode);
}
} catch (Exception e) {
    e.printStackTrace();
}
return responseBody;
}

```

```

@ExceptionHandler(NhBaseException.class)
public ResponseEntity<NH_NIS_ERR_DTO> errorProcessor(NhBaseException e) {
    NH_NIS_ERR_DTO errorDto = new NH_NIS_ERR_DTO();
    errorDto.setErrMethodName(e.getErrMethodName());
    errorDto.setStdErrCode(e.getStdErrCode());
    errorDto.setErrFileName(e.getErrFileName());
    errorDto.setErrClassName(e.getErrClassName());
    errorDto.setErrLineNo(e.getPgmLineNo());
    if (e.getAddMsgContents() != null) {
        errorDto.setAddMsgContents(e.getAddMsgContents());
    }
    if (e.getErrMsgType() == null) {
        errorDto.setErrType(TYPE.RUNTIME.name());
    } else {
        errorDto.setErrType(e.getErrMsgType().name());
    }
    if (e.getStackTrace() != null) {
        errorDto.setStackTrace(getTop15StackStrace(e.getStackTrace()));
    }
    if (e.getStdErrMsgContents() == null) {
        switch (e.getErrMsgType()) {
            case TYPE.RUNTIME, TYPE.COMMON, TYPE.AUTH -> {
                String template = SpringContext.getProperty(PREFIX +
e.getStdErrCode());
                String errorMessage = MessageFormat.format(template,
e.getMessageValue());
                errorDto.setStdErrMsgCntn(errorMessage);
            }
            case TYPE.SERVICE, TYPE.BIZ -> {
                // DB 조회 후 데이터 DB 버전으로 세팅
                String errorMessage =
messageCache.getMessage(e.getStdErrCode(), e.getMessageValue());
                errorDto.setStdErrMsgCntn(errorMessage);
            }
        }
    }
}

```

```

    } else {
        errorDto.setStdErrMsgCntn(e.getStdErrMsgContents());
    }
    return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(errorDto);
}

private List<String> getTop15StackStrace(StackTraceElement[] stackTrace) {
    return Arrays.stream(stackTrace).limit(15)
        .map(StackTraceElement::toString).collect(Collectors.toList());
}

/**
 * <PRE>
 * JSON 트리를 재귀적으로 순회하며 dtoLogicalName, fieldPropertyMap, 그리고 *List/*Array 로
 * 끝나는 필드를 제거
 * - ObjectNode: 필드 제거 후 하위 필드 재귀 처리
 * - ArrayNode: 배열 요소별 재귀 처리
 * </PRE>
 *
 * @param node 처리할 JSON 노드
 */
@SuppressWarnings("deprecation")
private void removeUnwantedFields(com.fasterxml.jackson.databind.JsonNode node) {
    if (node == null || node.isNull()) {
        return;
    }

    if (node.isObject()) {
        ObjectNode objectNode = (ObjectNode) node;

        // 불필요한 필드 제거
        objectNode.remove(DTO_LOGICAL_NAME);
        objectNode.remove(FIELD_PROPERTY_MAP);

        // *List 와 *Array 로 끝나는 필드 제거 (ConcurrentModificationException 방지를
        // 위해 먼저 리스트로 수집)
        List<String> fieldsToRemove = new java.util.ArrayList<>();
        Iterator<String> fieldNames = objectNode.fieldNames();
        while (fieldNames.hasNext()) {
            String fieldName = fieldNames.next();
            if (fieldName.endsWith("List") || fieldName.endsWith("Array")) {
                fieldsToRemove.add(fieldName);
            }
        }
        fieldsToRemove.forEach(objectNode::remove);

        // 하위 필드 재귀 처리
        Iterator<Map.Entry<String, com.fasterxml.jackson.databind.JsonNode>> fields =
        objectNode.fields();
        while (fields.hasNext()) {
            Map.Entry<String, com.fasterxml.jackson.databind.JsonNode> entry =
            fields.next();
            removeUnwantedFields(entry.getValue());
        }
    } else if (node.isArray()) {
        // 배열 요소별 재귀 처리
        for (com.fasterxml.jackson.databind.JsonNode element : node) {
            removeUnwantedFields(element);
        }
    }
}

```



```

}
/*****
* Copyright 2026 by Nonghyup. All rights reserved. Nonghyup 의 사전 승인 없이
* 본 내용의 전부 또는 일부에 대한 복사, 배포, 사용을 금합니다. Nonghyup의 사전 승인
* 없이 소스코드를 변경하여 사용하는 경우 소스코드에 대한 품질과 성능을 보장하지 않습니다.
*
* If you modify this source without Nonghyup's approval. Nonghyup does
* not guarantee the quality and performance of source.
*****/
/*****
* 이 력 사 항
*****/
* 일 자 버 전 작성자 변경사항
*****/
* 2026. 7. 15. 1.0 홍길동 최초작성
*****/
package nhnis.fw.common.transaction;

import org.springframework.beans.BeansException;
import org.springframework.context.ApplicationContext;
import org.springframework.context.ApplicationContextAware;
import org.springframework.stereotype.Component;
import org.springframework.transaction.PlatformTransactionManager;

/**
 * <PRE>
 * 설명
 *
 * </PRE>
 *
 * @author 홍길동
 * @version 1.0, 2026. 7. 15.
 * @logicalName
 */
@Component
public class InitTransactionManager implements ApplicationContextAware {

    private static ApplicationContext applicationContext;

    public static PlatformTransactionManager initTransactionManager(String transactionName) {
        PlatformTransactionManager transactionManager = (PlatformTransactionManager)
        applicationContext.getBean(transactionName);
        TransactionManagerHolder transactionManagerHolder = new
        TransactionManagerHolder(transactionManager);
        ServiceTransactionManager.setInstance(transactionManagerHolder);
        return transactionManager;
    }

    @Override
    public void setApplicationContext(ApplicationContext applicationContext) throws BeansException {
        this.applicationContext = applicationContext;
    }
}

package nhnis.fw.common.transaction;

public class ServiceTransactionManager {

    private static ThreadLocal<TransactionManagerHolder> transactionManager = new
    ThreadLocal<>();

    public static void setInstance(TransactionManagerHolder holder) {

```

```

        transactionManager.set(holder);
    }

    public static TransactionManagerHolder getInstance() {
        return transactionManager.get();
    }

    public static void remove() {
        TransactionManagerHolder txHolder = transactionManager.get();
        if (txHolder != null) {
            txHolder.release();
        }
        transactionManager.remove();
    }
}

package nhnis.fw.common.transaction;

import java.util.Deque;
import java.util.HashMap;
import java.util.LinkedList;
import java.util.Map;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.transaction.PlatformTransactionManager;
import org.springframework.transaction.TransactionDefinition;
import org.springframework.transaction.TransactionStatus;
import org.springframework.transaction.support.DefaultTransactionDefinition;

public class TransactionManagerHolder {

    private final Map<String, TransactionDefinition> definitionMap = new HashMap<>();
    private final Deque<TransactionStatus> txStack = new LinkedList<>();
    private final PlatformTransactionManager transactionManager;

    private static final String DEFAULT_DEFINITION = "defaultDefinition";

    public TransactionManagerHolder(PlatformTransactionManager transactionManager) {
        this.transactionManager = transactionManager;
    }

    public PlatformTransactionManager getTransactionManager() {
        return this.transactionManager;
    }

    public TransactionStatus beginTransaction() {
        TransactionStatus status = beginTransaction(DEFAULT_DEFINITION);
        return status;
    }

    public TransactionStatus beginTransaction(String definitionName) {
        TransactionDefinition txDefinition = this.definitionMap.get(definitionName);
        if (txDefinition == null) {
            return beginTransaction(definitionName,
TransactionDefinition.PROPROPAGATION_REQUIRES_NEW,
TransactionDefinition.ISOLATION_DEFAULT);
        } else {
            return beginTransaction(definitionName);
        }
    }
}

```



```

    }

    public TransactionStatus beginTransaction(String definitionName, int propagationStrategy, int
isolationLevel) {
        TransactionDefinition txDefinition = getDefaultTransactionDefinition(definitionName,
propagationStrategy,
            isolationLevel);
        return beginTransaction(txDefinition);
    }

    public TransactionStatus beginTransaction(TransactionDefinition txDefinition) {
        TransactionStatus txStatus = this.transactionManager.getTransaction(txDefinition);
        this.txStack.add(txStatus);
        return txStatus;
    }

    private TransactionDefinition getDefaultTransactionDefinition(String definitionName, int
propagationStrategy,
        int isolationLevel) {
        int isoLevel = -1;
        DefaultTransactionDefinition definition = (DefaultTransactionDefinition)
this.definitionMap.get(definitionName);
        if (definition == null) {
            definition = new DefaultTransactionDefinition();
            definition.setName(definitionName);
            definition.setPropagationBehavior(propagationStrategy);
            if (isolationLevel == 0) {
                definition.setIsolationLevel(isoLevel);
            } else {
                definition.setIsolationLevel(isolationLevel);
            }
            definition.setIsolationLevel(isolationLevel);
            this.definitionMap.putIfAbsent(definitionName, definition);
            definition =
(DefaultTransactionDefinition)this.definitionMap.get(definitionName);
        }
        return (TransactionDefinition) definition;
    }

    public void commit() {
        TransactionStatus currentTx = this.txStack.pop();
        this.transactionManager.commit(currentTx);
    }

    public void commit(TransactionStatus txStatus) {
        this.transactionManager.commit(txStatus);
        this.txStack.remove(txStatus);
    }

    public void rollback() {
        TransactionStatus currentTx = this.txStack.pop();
        this.transactionManager.rollback(currentTx);
    }

    public void rollback(TransactionStatus txStatus) {
        this.transactionManager.rollback(txStatus);
        this.txStack.remove(txStatus);
    }

    public void release() {
        this.definitionMap.clear();
        this.txStack.clear();
    }

```

```

    }
}
package nhnis.fw.common.util;

import java.nio.charset.StandardCharsets;
import java.util.Base64;

import javax.crypto.Cipher;
import javax.crypto.spec.IvParameterSpec;
import javax.crypto.spec.SecretKeySpec;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class AES256Util {

    private static Logger LOGGER = LoggerFactory.getLogger(AES256Util.class);

    private static final String ALG= "AES/CBC/PKCS5Padding";
    private static final String AES = "AES";

    private String iv;

    public String encrypt(String key, String plainText) throws Throwable {
        try {
            this.iv = key.substring(0, 16);
            Cipher cipher = Cipher.getInstance(ALG);
            SecretKeySpec keySpec = new SecretKeySpec(this.iv.getBytes(), AES);
            IvParameterSpec spec = new IvParameterSpec(this.iv.getBytes());
            cipher.init(1, keySpec, spec);
            byte[] encrypt =
cipher.doFinal(plainText.getBytes(StandardCharsets.UTF_8));
            return Base64.getEncoder().encodeToString(encrypt);
        } catch (Exception e) {
            LOGGER.error("[AES256Util] Encrypt fail: {}", e.getMessage());
            throw e;
        }
    }

    public String decrypt(String key, String cipherText) throws Exception {
        try {
            this.iv = key.substring(0, 16);
            Cipher cipher = Cipher.getInstance(ALG);
            SecretKeySpec keySpec = new SecretKeySpec(this.iv.getBytes(), AES);
            IvParameterSpec spec = new IvParameterSpec(this.iv.getBytes());
            cipher.init(2, keySpec, spec);
            byte[] decodeBytes = Base64.getDecoder().decode(cipherText);
            byte[] decrypt = cipher.doFinal(decodeBytes);
            return new String(decrypt, StandardCharsets.UTF_8);
        } catch (Exception e) {
            LOGGER.error("[AES256Util] Decrypt fail: {}", e.getCause());
            throw e;
        }
    }
}
package nhnis.fw.common.util;

import java.math.BigDecimal;
import java.text.NumberFormat;

import org.slf4j.Logger;

```



```

import org.slf4j.LoggerFactory;

public class BigDecimalUtil {

    protected static final Logger LOGGER =
LoggerFactory.getLogger(BigDecimalUtil.class);

    /**
     * 설명 : 설명 : BigDecimal Object 를 반환한다.
     * null 또는 "" 값인 경우 0 값을 반환한다. (scale 설정 없음)
     * ex)
     * newBigDecimal("1701677800.82"); => 1701677800.82
     * newBigDecimal("1.7016778E9"); => 1701677800
     * newBigDecimal(null); => 0
     * newBigDecimal(""); => 0
     * @param value
     * @return BigDecimal
     */

    public static BigDecimal newBigDecimal(Object value) {

        return new BigDecimal(unExponential(defaultString(value, "0")));
    }

    /**
     * 설명 : BigDecimal Object 를 반환한다.
     * null 또는 "" 값인 경우 0 값을 반환한다.
     * ex)
     * newBigDecimal("1701677800.82", 0, BigDecimal.ROUND_DOWN); => 1701677800
     * newBigDecimal("1.7016778E9", 0, BigDecimal.ROUND_DOWN); => 1701677800
     * newBigDecimal("1701677800.82", 1, BigDecimal.ROUND_DOWN); => 1701677800.8
     * newBigDecimal(null, 1, BigDecimal.ROUND_DOWN); => 0
     * newBigDecimal("", 1, BigDecimal.ROUND_DOWN); => 0
     * @param value
     * @param scale
     * @param round
     * @return BigDecimal
     */

    public static BigDecimal newBigDecimal(Object value, int scale, int round) {

        return new BigDecimal(unExponential(defaultString(value, "0"))).setScale(scale,
round);
    }

    /**
     * 설명 : 지수 표시된 수를 정수/실수 형태로 변경
     * null 또는 "" 값인 경우 NumberFormatException 이 발생한다.
     * ex)
     * unExponential("1.7016778E9"); => "1701677800"
     * unExponential(null); => NumberFormatException
     * unExponential(""); => NumberFormatException
     * @param value
     * @return String
     */

    public static String unExponential(Object value) {

        String number = String.valueOf(value);

        NumberFormat formater = NumberFormat.getInstance();
        formater.setGroupingUsed(false);

        number = formater.format(new BigDecimal(number).doubleValue());
    }
}

```